Inserting a node in LinkedList

```
/*Structure of the linked list node is as
struct Node {
  int data;
  struct Node * next;
  Node(int x) {
    data = x;
    next = NULL;
  }
}; */

class Solution {
  public:
    Node *insertAtEnd(Node *head, int x) {
        // Code here

        //find the last node
        struct Node* temp = head;
        struct Node* newNode = new Node(x);

        if(head == NULL){
            return new Node(x);
        }

        while(temp->next != NULL){
            temp = temp->next;
        }

        temp->next = newNode;

        return head;
    }
};
```

Deleting node in LL

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    void deleteNode(ListNode* node) {
        if (node->next){
            node->val = node->next->val;
            node->next = node->next->next;
        }
        else node = NULL;
    }
};
```

Find length of LL

```
/* Link list node */
/*
struct Node
{
    int data;
    Node* next;
    Node(int x) {  data = x;  next = NULL; }
}; */

class Solution {
  public:
    // Function to count nodes of a linked list.
    int getCount(struct Node* head) {

        // Code here
        int count= 0;

        Node* temp = head;
        while(temp!=NULL){
            temp = temp->next;
            count++;
        }
        return count;
    }
};
```

Search an element in LL

```
/* Link list node */
/*
struct Node
{
    int data;
    Node* next;
    Node(int x) {  data = x;  next = NULL; }
}; */

class Solution {
  public:
    // Function to count nodes of a linked list.
    bool searchKey(int n, struct Node* head, int key) {
        // Code here
        Node *temp=head;
        while(temp){
            if(temp->data==key){
                return true;
            }
            else{
                temp=temp->next;
            }
        }
        return false;
    }
};
```

Doubly Linkedlist

Insert a node in doubly LL

```
/* a Node of the doubly linked list
struct Node
{
  int data;
  struct Node *next;
  struct Node *prev;
  Node(int x) { data = x; next = prev = NULL; }
}; */

//Function to insert a new node at given position in doubly linked list.
void addNode(Node *head, int pos, int data)
{
   Node * current = head;
   int count=0;

   while(current != NULL && count<pos){
       count++;
       current=current->next;
   }

   Node*newNode = new Node(data);

   newNode->next=current->next;

   if(current->next!=NULL){
       current->next->prev=newNode;
   }

   newNode->prev = current;
   current->next = newNode;
}
```

Delete a node form DLL

```
/* Structure of Node
struct Node
{
  int data;
  struct Node *next;
  struct Node *prev;

  Node(int x){
      data = x;
      next = NULL;
      prev = NULL;
  }

};
*/

class Solution {
  public:
    Node* deleteNode(Node* head, int x)
    {
        if(x == 1)
        {
            Node *p = head->next, *temp = head;
            p->prev = NULL;
            head = p;
            delete temp;
            return head;
        }
        else
        {
            Node *p1 = head, *p2 = head->next, *temp;
            int i=1;
            while(i<x-1)
            {
                p1 = p1->next;
                p2 = p2->next;
                i++;
            }
            p1->next = p2->next;
```

```
            temp = p2;
            p2 = p2->next;
            if(p2 != NULL)  p2->prev = p1;
            delete temp;
            return head;
        }
    }
};
```

Reverse a DLL

```
/*
struct Node
{
    int data;
    Node * next;
    Node * prev;
    Node (int x)
    {
        data=x;
        next=NULL;
        prev=NULL;
    }

};
*/
class Solution
{
    public:
    Node* reverseDLL(Node * head)
    {
        struct Node *pr,*temp,*t;
        temp=head;

        while(temp->next){

            t=temp->next;
            temp->next=temp->prev;
            temp->prev=t;

            pr=temp;
            temp=temp->prev;

        }
        head=temp;
        temp->next=pr;
        temp->prev=NULL;
        return head;
    }
};
```

Medium Problems of LL

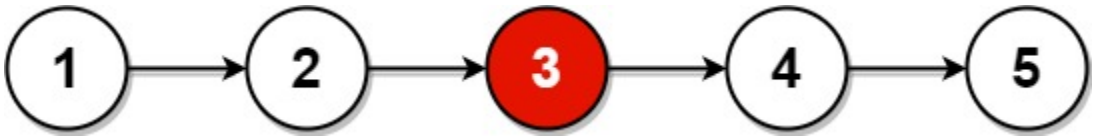
Middle of a LL

https://leetcode.com/problems/middle-of-the-linked-list

Given the `head` of a singly linked list, return *the middle node of the linked list*.

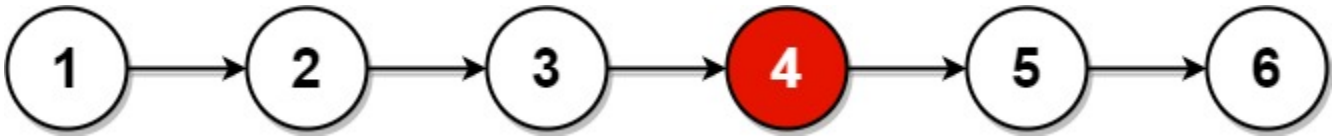
If there are two middle nodes, return **the second middle** node.

Example 1:



Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.

Example 2:



Input: head = [1,2,3,4,5,6]
Output: [4,5,6]
Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.

```
ListNode* middleNode(ListNode* head) {
        ListNode *slow = head, *fast = head;
        while (fast && fast->next)
            slow = slow->next, fast = fast->next->next;
        return slow;
    }
```

Reverse a LL

Iterative

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode *nextNode, *prevNode = NULL;
        while (head) {
            nextNode = head->next;
            head->next = prevNode;
            prevNode = head;
            head = nextNode;
        }
        return prevNode;
    }
};
```

Recursive

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if (!head || !(head -> next)) {
            return head;
        }
        ListNode* node = reverseList(head -> next);
        head -> next -> next = head;
        head -> next = NULL;
        return node;
    }
};
```

Detect a loop in LL

https://leetcode.com/problems/linked-list-cycle/

```cpp
class Solution {
public:
    bool hasCycle(ListNode *head) {
        std::unordered_set<ListNode*> visited_nodes;
        ListNode *current_node = head;
        while (current_node != nullptr) {
            if (visited_nodes.find(current_node) != visited_nodes.end()) {
                return true;
            }
            visited_nodes.insert(current_node);
            current_node = current_node->next;
        }
        return false;
    }
};
```

Find the starting point in LL

https://leetcode.com/problems/linked-list-cycle-ii/

```
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        // edge case - empty list
        if (!head || !head->next || !head->next->next) return NULL;
        // support animals
        ListNode *turtle = head, *hare = head;
        // checking if we loop or not
        while (hare->next && hare->next->next) {
            hare = hare->next->next;
            turtle = turtle->next;
            if (hare == turtle) break;
        }
        // exiting if we do not find a loop
        if (hare != turtle) return NULL;
        // finding the start of the loop
        turtle = head;
        while (turtle != hare) {
            hare = hare->next;
            turtle = turtle->next;
        }
        return turtle;
    }
};
```

Length of a loop in LL

https://www.geeksforgeeks.org/problems/find-length-of-loop/1

```
//Function to find the length of a loop in the linked list.
int findlen(struct Node *slow,struct Node *fast){
    int ct=1;
    fast=fast->next;
    while(fast!=slow){
        fast=fast->next;
        ct++;
    }
    return ct;

}
int countNodesinLoop(struct Node *head)
{

         Node* fast=head;
         Node* slow=head;
        while(fast!=NULL && fast->next!=NULL){
            slow=slow->next;
            fast=fast->next->next;
            if(slow==fast) return findlen(slow,fast);
        }
        return 0;
}
```

Check if a LL is palindrome or not

```cpp
class Solution {
public:
    bool isPalindrome(ListNode* head) {
        // Steps to follow:
        // 1_) Find the middle element
        ListNode *slow = head, *fast = head;
        while(fast!=NULL && fast->next !=NULL){
            slow = slow->next;
            fast = fast->next->next;
        }
        // 2_) if the no of nodes are odd then move slow to one point
        if(fast != NULL && fast->next == NULL){
            slow = slow->next;
        }
        //3_) Reverse the end half
        ListNode *prev = NULL;
        ListNode *temp = NULL;
        while(slow != NULL && slow->next != NULL){
            temp = slow->next;
            slow->next = prev;
            prev = slow;
            slow = temp;
        }
        if(slow != NULL){
            slow->next = prev;
        }
        //4_) Compare the start half and end half if found any inequality then return false otherw
        fast = head;
        while(slow && fast){
            if(slow->val != fast->val){
                return false;
            }
            slow = slow->next;
            fast = fast->next;
        }
        return true;

    }
};
```

Segregate odd and even nodes in LL

https://leetcode.com/problems/odd-even-linked-list/

```
class Solution {
public:
    ListNode* oddEvenList(ListNode* head) {
        if(!head || !head->next || !head->next->next) return head;

        ListNode *odd = head;
        ListNode *even = head->next;
        ListNode *even_start = head->next;

        while(odd->next && even->next){
            odd->next = even->next; //Connect all odds
            even->next = odd->next->next;  //Connect all evens
            odd = odd->next;
            even = even->next;
        }
        odd->next = even_start;   //Place the first even node after the last odd node.
        return head;
    }
};
```

Remove Nth node from the back of LL

https://leetcode.com/problems/remove-nth-node-from-end-of-list

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode* dummy = new ListNode(0);
        dummy->next = head;
        ListNode* first = dummy;
        ListNode* second = dummy;

        for (int i = 0; i <= n; ++i) {
            first = first->next;
        }

        while (first != nullptr) {
            first = first->next;
            second = second->next;
        }

        ListNode* temp = second->next;
        second->next = second->next->next;
        delete temp;

        return dummy->next;
    }
};
```

Delete the middle Node of LL

https://leetcode.com/problems/delete-the-middle-node-of-a-linked-list

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* deleteMiddle(ListNode* head) {
        if(head == NULL)return NULL;
        ListNode* prev = new ListNode(0);
        prev->next = head;
        ListNode* slow = prev;
        ListNode* fast = head;
        while(fast != NULL && fast->next != NULL){
            slow = slow->next;
            fast = fast->next->next;
        }
        slow->next = slow->next->next;
        return prev->next;
    }
};
```

Sort the LL

https://leetcode.com/problems/sort-list/

QuickSort

```
ListNode* sortList(ListNode* head, ListNode* tail = nullptr)
{
  if (head != tail) {
    // Use head node as the pivot node
    // Everything in the _smaller_ list will be less than _head_
    // and everything appearing after _head_ in the list is greater or equal
    ListNode* smaller;
    ListNode** smaller_next = &smaller;
    for (ListNode** prev = &head->next; *prev != tail; ) {
      if (head->val > (**prev).val) {
        *smaller_next = *prev;
        smaller_next = &((**smaller_next).next);

        // Remove smaller node from original list
        *prev = (**prev).next;
      } else {
        // Correct position, skip over
        prev = &((**prev).next);
      }
    }

    // Connect the end of smaller list to the head (which is the partition node)
    // We now have. [smaller list...] -> head -> [larger list]
    *smaller_next = head;

    // Continue to sort everything after head
    head->next = sortList(head->next, tail);

    // Sort everything upto head
    head = sortList(smaller, head);
  }
  return head;
}
```

Sort LL of 0's 1's and 2's

https://bit.ly/3Ceotvr

```
/*

  Node is defined as
  struct Node {
    int data;
    struct Node *next;
    Node(int x) {
        data = x;
        next = NULL;
    }
};

*/
class Solution
{
    public:
    //Function to sort a linked list of 0s, 1s and 2s.
    Node* segregate(Node *head) {
        // EDGE CASE : only 0/1 element
        if(head == NULL || head->next == NULL) return head;

        Node* zeroHead = new Node(-1);
        Node* oneHead = new Node(-1);
        Node* twoHead = new Node(-1);

        Node* zero = zeroHead;
        Node* one = oneHead;
        Node* two = twoHead;

        Node* temp = head;
        while(temp != NULL) {
            if(temp->data == 0) {
                zero->next = temp;
                zero = temp;
            }
            else if(temp->data == 1) {
                one->next = temp;
                one = temp;
            }
```

```
            else {
                two->next = temp;
                two = temp;
            }
            temp = temp->next;
        }

        zero->next = (oneHead->next) ? (oneHead->next) : (twoHead->next);
        one->next = twoHead->next;
        two->next = NULL;

        Node* newHead = zeroHead->next;

        delete zeroHead;
        delete oneHead;
        delete twoHead;

        return newHead;
    }
};
```

Find intersection point of Y LL

https://leetcode.com/problems/intersection-of-two-linked-lists

```
ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
    ListNode *ptrA = headA, *ptrB = headB;
    while (ptrA != ptrB) {
        ptrA = ptrA ? ptrA->next : headB;
        ptrB = ptrB ? ptrB->next : headA;
    }
    return ptrA;
}
```

Add 1 to a number represented by LL

https://www.geeksforgeeks.org/problems/add-1-to-a-number-represented-as-linked-list/1

```
//User function template for C++

/*

struct Node
{
    int data;
    struct Node* next;

    Node(int x){
        data = x;
        next = NULL;
    }
};

*/

class Solution
{
    public:

    Node* reverse(Node* head) {
        Node*curr=head;
        Node*prev=NULL;
        Node*forw=NULL;
        while(curr!=NULL)
        {
            forw=curr->next;
            curr->next=prev;
            prev=curr;
            curr=forw;
        }
        head=prev;
        return head;
    }

    Node* addOne(Node *head)
    {
        head=reverse(head);
```

```
        Node*temp=head;
        Node*dummy=new Node(0);
        Node*temp2=dummy;
        int carry=1;
        while(temp!=NULL || carry)
        {
            int sum=carry;
            if(temp!=NULL)
            {
                sum+=temp->data;
                temp=temp->next;
            }
            int node_val=sum%10;
            carry=sum/10;
            temp2->next=new Node(node_val);
            temp2=temp2->next;
        }
        dummy=dummy->next;
        dummy=reverse(dummy);
        return dummy;
        // Your Code here
        // return head of list after adding one
    }
};
```

Add 2 Numbers in LL

https://leetcode.com/problems/add-two-numbers/

```
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode* dummyHead = new ListNode(0);
        ListNode* tail = dummyHead;
        int carry = 0;

        while (l1 != nullptr || l2 != nullptr || carry != 0) {
            int digit1 = (l1 != nullptr) ? l1->val : 0;
            int digit2 = (l2 != nullptr) ? l2->val : 0;

            int sum = digit1 + digit2 + carry;
            int digit = sum % 10;
            carry = sum / 10;

            ListNode* newNode = new ListNode(digit);
            tail->next = newNode;
            tail = tail->next;

            l1 = (l1 != nullptr) ? l1->next : nullptr;
            l2 = (l2 != nullptr) ? l2->next : nullptr;
        }

        ListNode* result = dummyHead->next;
        delete dummyHead;
        return result;
    }
};
```

Medium Problems of LL

Delete all occurences of a key in DLL

https://bit.ly/3zuBr66

You are given the **head_ref** of a doubly Linked List and a **Key**. Your task is to **delete all occurrences** of the given key if it is present and return the new DLL.

Example1:

Input:

2<->2<->10<->8<->4<->2<->5<->2

2

Output:

10<->8<->4<->5

Explanation:

All Occurences of 2 have been deleted.

Example2:

Input:

9<->1<->3<->4<->5<->1<->8<->4

9

Output:

1<->3<->4<->5<->1<->8<->4

Explanation:

All Occurences of 9 have been deleted.

```
/ User function Template for C++

/* a Node of the doubly linked list
struct Node
{
  int data;
  struct Node *next;
  struct Node *prev;
  Node(int x) { data = x; next = prev = NULL; }
}; */

class Solution {
  public:
    void deleteAllOccurOfX(struct Node** head_ref, int x) {
    if (head_ref == NULL || *head_ref == NULL) {
        return;
    }

    struct Node* temp = *head_ref;

    while (temp != NULL) {
        if (temp->data == x) {
            struct Node* to_delete = temp;
            if (temp == *head_ref) {
                *head_ref = temp->next;
                if (*head_ref != NULL) {
                    (*head_ref)->prev = NULL;
                }
            } else {
                if (temp->prev) {
                    temp->prev->next = temp->next;
                }
                if (temp->next) {
                    temp->next->prev = temp->prev;
                }
            }
            temp = temp->next;
            free(to_delete);
        } else {
```

```
            temp = temp->next;
        }
    }
}
};
```

Two Sum in DLL

https://www.geeksforgeeks.org/problems/find-pairs-with-given-sum-in-doubly-linked-list/1
Given a sorted doubly linked list of positive distinct elements, the task is to find pairs in a doubly-linked list whose sum is equal to given value **target**.

Example 1:

Input:
1 <-> 2 <-> 4 <-> 5 <-> 6 <-> 8 <-> 9
target = 7
Output: (1, 6), (2,5)
Explanation: We can see that there are two pairs
(1, 6) and (2,5) with sum 7.

Example 2:

Input:
1 <-> 5 <-> 6
target = 6
Output: (1,5)
Explanation: We can see that there is one pairs (1, 5) with sum 6.

```cpp
//User function Template for C++

/* Doubly linked list node class
class Node
{
public:
    int data;
    Node *next, *prev;
    Node(int val) : data(val), next(NULL), prev(NULL)
    {
    }
};
*/

class Solution
{
public:

   Node* findTail(Node* head){
       Node* temp = head;
       while(temp->next!=NULL ){
           temp = temp->next;
       }
       return temp;
   }

    vector<pair<int, int>> findPairsWithGivenSum(Node *head, int target)
    {
         vector<pair<int, int>> ans;
         if(head==NULL) return ans;
         Node* temp1 = head;
         Node* temp2 = findTail(head);
         while(temp1->data < temp2->data){
            if(temp1->data + temp2->data == target){
                ans.push_back({temp1->data , temp2->data});
                temp1=temp1->next;
                temp2=temp2->prev;
            }
            else if (temp1->data + temp2->data < target) temp1=temp1->next;
```

```
            else if (temp1->data + temp2->data > target)  temp2=temp2->prev;

        }
        return ans;
    }
};
```

Remove Duplicates in Sorted DLL

https://bit.ly/3FtJUtZ

Given a doubly linked list of **n** nodes sorted by values, the task is to remove duplicate nodes present in the linked list.

Example 1:

Input:
n = 6
1<->1<->1<->2<->3<->4
Output:
1<->2<->3<->4
Explanation:
Only the first occurance of node with value 1 is
retained, rest nodes with value = 1 are deleted.

Example 2:

Input:
n = 7
1<->2<->2<->3<->3<->4<->4
Output:
1<->2<->3<->4
Explanation:
Only the first occurance of nodes with values 2,3 and 4 are
retained, rest repeating nodes are deleted.

```
/*
struct Node
{
    int data;
    Node * next;
    Node * prev;
    Node (int x)
    {
        data=x;
        next=NULL;
        prev=NULL;
    }

};
*/

class Solution
{
public:

    Node * removeDuplicates(struct Node *head)
    {
        Node* temp = head;
        while (temp != NULL && temp->next != NULL){
            Node* nextnode = temp->next;
            while (nextnode != NULL && temp->data == nextnode->data){
                Node* duplicate = nextnode;
                nextnode = nextnode->next;
                free(duplicate);
            }
            temp->next = nextnode;
            if (nextnode != NULL) nextnode->prev = temp;
            temp = temp->next;
        }
        return head;
    }
};
```

Hard Problems of LL

Reverse LL in a group of given size K

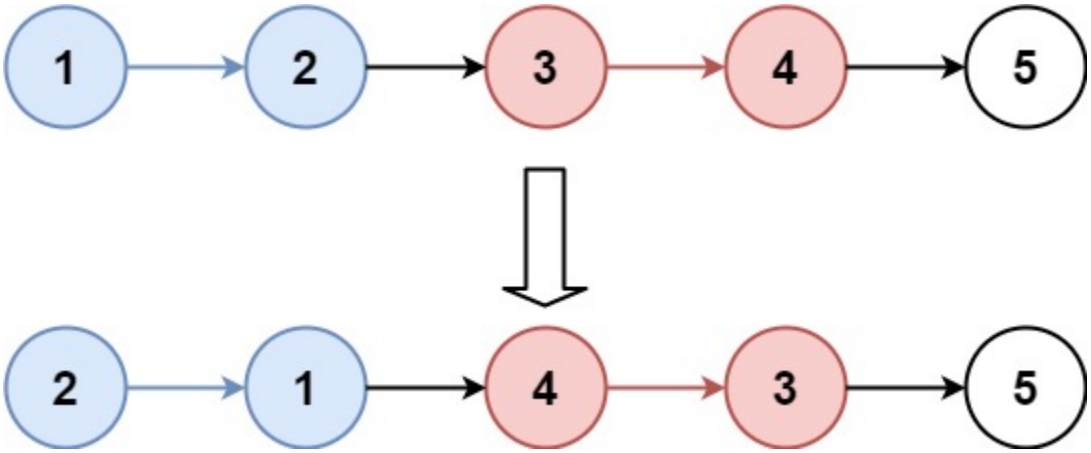
https://leetcode.com/problems/reverse-nodes-in-k-group

Given the `head` of a linked list, reverse the nodes of the list `k` at a time, and return *the modified list*.

`k` is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of `k` then left-out nodes, in the end, should remain as it is.

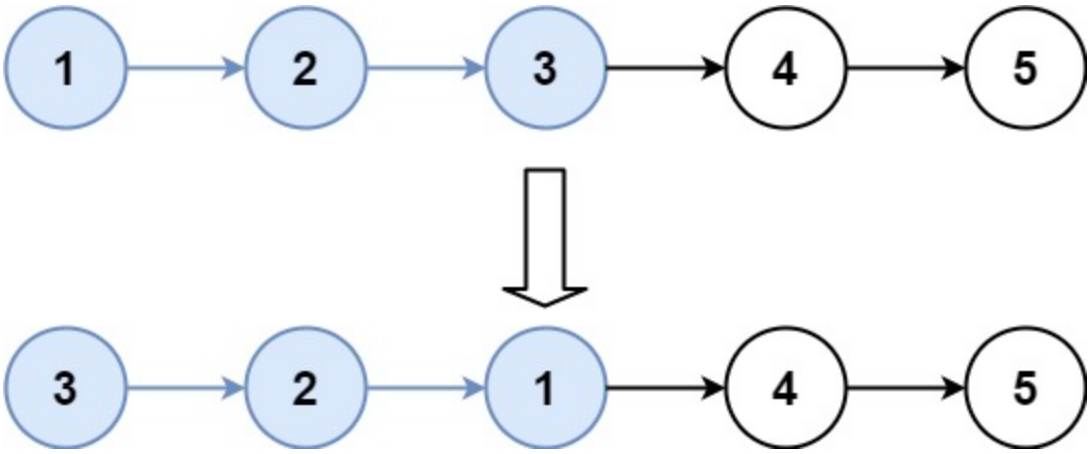
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]

Example 2:



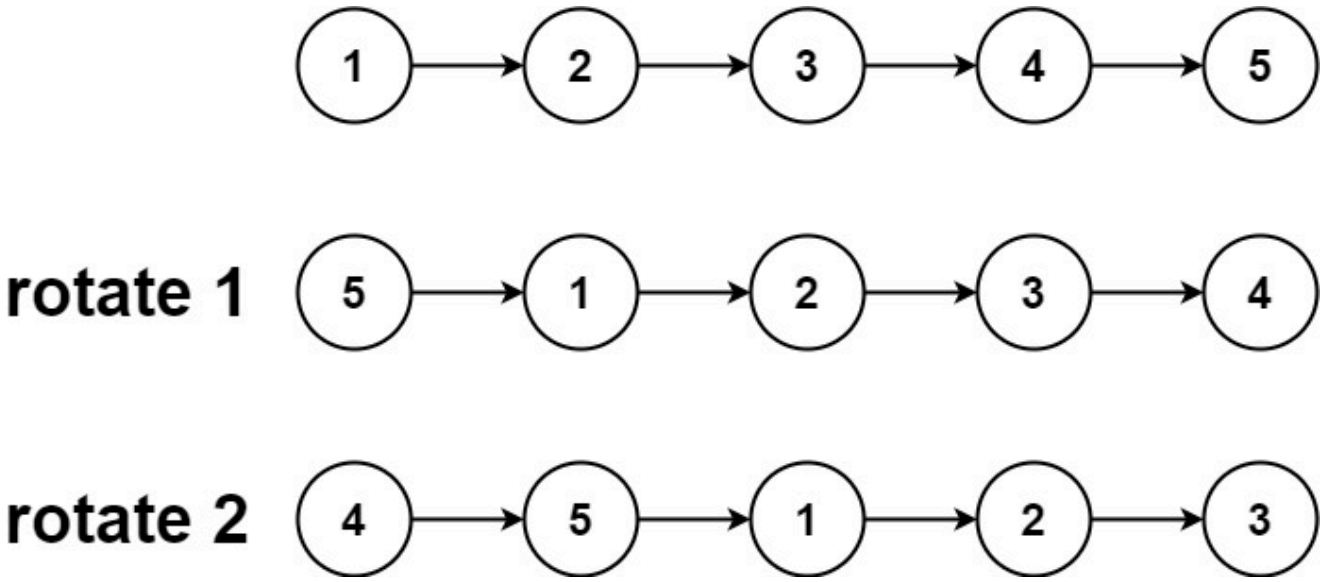
Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]

```
ListNode* reverseKGroup(ListNode* head, int k) {
        ListNode* cursor = head;
        for(int i = 0; i < k; i++){
            if(cursor == nullptr) return head;
            cursor = cursor->next;
        }
        ListNode* curr = head;
        ListNode* prev = nullptr;
        ListNode* nxt = nullptr;
        for(int i = 0; i < k; i++){
            nxt = curr->next;
            curr->next = prev;
            prev = curr;
            curr = nxt;
        }
        head->next = reverseKGroup(curr, k);
        return prev;
    }
```

Rotate a LL

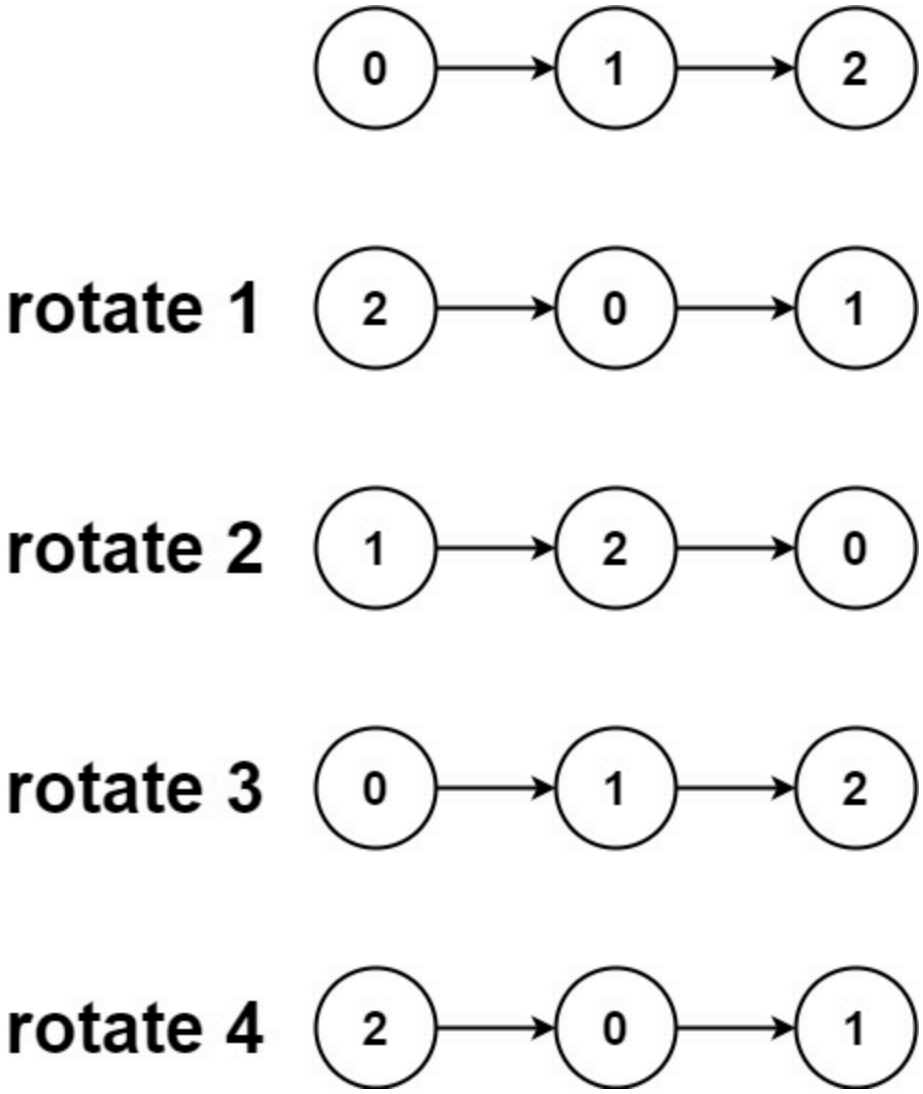
https://leetcode.com/problems/rotate-list
Given the `head` of a linked list, rotate the list to the right by `k` places.

Example 1:



Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]

Example 2:



Input: head = [0,1,2], k = 4

Output: [2,0,1]

```
class Solution {
public:
    ListNode* rotateRight(ListNode* head, int k) {
        if(!head) return head;
        int len=1; // number of nodes
        ListNode *newH, *tail;
        newH=tail=head;
        while(tail->next)  // get the number of nodes in the list
        {
            tail = tail->next;
            len++;
        }
        tail->next = head; // circle the link
        if(k %= len)
        {
            for(auto i=0; i<len-k; i++) tail = tail->next; // the tail node is the (len-k)-th node
        }
        newH = tail->next;
        tail->next = NULL;
        return newH;
    }
};
```

Flattening of LL

https://bit.ly/3w9TKf8

Given a Linked List of size n, where every node represents a sub-linked-list and contains two pointers:
(i) a **next** pointer to the next node,
(ii) a **bottom** pointer to a linked list where this node is head.
Each of the sub-linked-list is in sorted order.
Flatten the Link List such that all the nodes appear in a single level while maintaining the sorted order.

Note: The flattened list will be printed using the **bottom pointer** instead of the next pointer.
For more clarity have a look at the printList() function in the driver code.

Examples:

Input:
5 -> 10 -> 19 -> 28
| | | |
7 20 22 35
| | |
8 50 40
| |
30 45
Output: 5-> 7-> 8- > 10 -> 19-> 20-> 22-> 28-> 30-> 35-> 40-> 45-> 50.
Explanation: The resultant linked lists has every node in a single level.(**Note:** | represents the bottom pointer.)

Input:
5 -> 10 -> 19 -> 28
| |
7 22
| |
8 50
|
30
Output: 5-> 7-> 8-> 10-> 19-> 22-> 28-> 30-> 50
Explanation: The resultant linked lists has every node in a single level.(**Note:** | represents the bottom pointer.)

```
/* Node structure  used in the program

struct Node{
    int data;
    struct Node * next;
    struct Node * bottom;

    Node(int x){
        data = x;
        next = NULL;
        bottom = NULL;
    }

};
*/
Node *merge(Node* list1, Node* list2){
    Node* dummy = new Node(-1);
    Node* temp = dummy;
    while(list1 && list2){
        if(list1->data<list2->data){
            temp->bottom = list1;
            temp = list1;
            list1 = list1->bottom;
        }else{
            temp->bottom = list2;
            temp = list2;
            list2 = list2->bottom;
        }
        temp->next = NULL;
    }
    if(list1){
        temp->bottom = list1;
    }else{
        temp->bottom = list2;
    }
    return dummy->bottom;
}
Node *flatten(Node *root) {
    if(!root || !root->next){
```

```
        return root;
    }
    Node* mergeHead = flatten(root->next);
    root = merge(root,mergeHead);
    return root;
}
```

Clone a linked list with random and next pointer

https://leetcode.com/problems/copy-list-with-random-pointer/
A linked list of length `n` is given such that each node contains an additional random pointer, which could point to any node in the list, or `null`.

Construct a **deep copy** of the list. The deep copy should consist of exactly `n` **brand new** nodes, where each new node has its value set to the value of its corresponding original node. Both the `next` and `random` pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. **None of the pointers in the new list should point to nodes in the original list**.

For example, if there are two nodes `X` and `Y` in the original list, where `X.random --> Y`, then for the corresponding two nodes `x` and `y` in the copied list, `x.random --> y`.

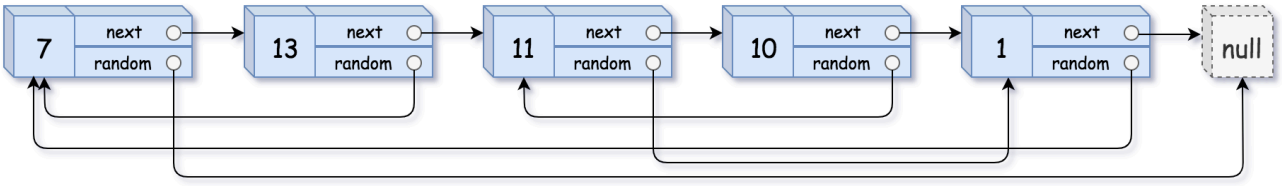
Return *the head of the copied linked list*.

The linked list is represented in the input/output as a list of `n` nodes. Each node is represented as a pair of `[val, random_index]` where:

- `val`: an integer representing `Node.val`
- `random_index`: the index of the node (range from `0` to `n-1`) that the `random` pointer points to, or `null` if it does not point to any node.

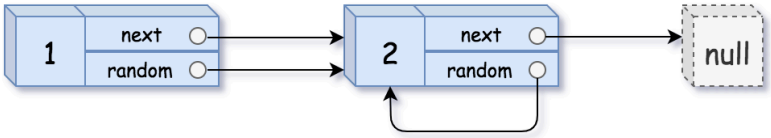
Your code will **only** be given the `head` of the original linked list.

Example 1:



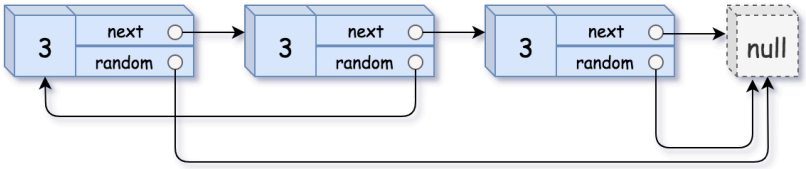
Input: head = 7,null],[13,0],[11,4],[10,2],[1,0
Output: 7,null],[13,0],[11,4],[10,2],[1,0

Example 2:



Input: head = 1,1],[2,1
Output: 1,1],[2,1

Example 3:



Input: head = 3,null],[3,0],[3,null
Output: 3,null],[3,0],[3,null

```
/*
// Definition for a Node.
class Node {
public:
    int val;
    Node* next;
    Node* random;
    Node(int _val) {
        val = _val;
        next = NULL;
        random = NULL;
    }
};
*/
class Solution {
public:
    Node* copyRandomList(Node* head) {
        map<Node*, Node*> m;
        int i=0;
        Node* ptr = head;
        while (ptr) {
            m[ptr] =new Node(ptr->val);
            ptr = ptr->next;
        }
        ptr = head;
        while (ptr) {
            m[ptr]->next = m[ptr->next];
            m[ptr]->random = m[ptr->random];
            ptr = ptr->next;
        }
        return m[head];
    }
};
```